OWASP Juice Shop
Security Assessment Report

Six-challenge practical security assessment

Application	OWASP Juice Shop 20.3.0
Environment	Local instance on Windows
Assessment type	Hands-on vulnerability challenge assessment
Assessment date	November 2026
Prepared for	Personal cybersecurity portfolio

This report records the tests performed, the evidence retained, the results supported by that evidence, and recommended defensive controls. It does not claim successful exploitation where the retained evidence does not establish it.

1. Executive Summary

This assessment used the intentionally vulnerable OWASP Juice Shop 20.3.0 application in a local training environment. Six challenges were selected across SQL injection/database exposure, broken access control, cross-site scripting, authentication, and sensitive data exposure.

The retained evidence directly supports successful demonstrations for DOM-based XSS, Password Strength, and Exposed Credentials. Reflected XSS was observed during the practical session and HTML reflection is visible in the retained screenshot, but the JavaScript alert itself was not captured in the evidence pack. Database Schema was reported as completed, but its retained screenshot only shows the challenge card. View Basket investigation was completed, but no unauthorized cross-account access was demonstrated.

Evidence-led conclusion: three challenges have direct retained evidence of successful completion/execution (DOM XSS, Password Strength, Exposed Credentials); Reflected XSS has a reported alert and direct evidence of HTML reflection; Database Schema completion is user-reported but not independently verified by the retained screenshot; View Basket remains inconclusive under the recorded test conditions.

Assessment results at a glance

Challenge	Recorded result	Evidence strength
Database Schema	Completion reported; exploit result not shown	Limited
View Basket	Investigation completed; no exploit demonstrated	Investigation only
DOM XSS	JavaScript alert executed	Direct
Reflected XSS	HTML reflected; alert reported, not captured	Partial
Password Strength	Challenge success notification	Direct
Exposed Credentials	Client-side credentials found; success notification	Direct

The report is limited to the local training application and the tests recorded in the supplied assessment notes and evidence pack. It is not a production penetration test or a claim about the security of any external system.

2. Scope and Methodology

In scope: the local OWASP Juice Shop 20.3.0 instance and the six selected challenge areas listed in this report.

Out of scope: external systems, production accounts, third-party services, and any system for which authorization was not explicitly granted.

Approach: challenge-led manual testing using the application interface and browser developer tools. Activities recorded in the supplied notes include search-parameter testing, observing rendered page output, inspecting Network activity, inspecting event listeners, reviewing client-side JavaScript, and testing the challenge-provided lab credentials.

Environment: OWASP Juice Shop 20.3.0 running locally on a Windows machine. The test application was accessed through the local browser.

Assessment limitations

The evidence pack is not equally complete for all six challenges. Some screenshots establish that a challenge was selected or investigated but do not prove exploitation. The report therefore separates observed outcomes, user-reported outcomes, and unverified conclusions. No additional tests are represented as having been performed.

3. Findings and Challenge Assessments

3.1 Database Schema — SQL Injection / Database Exposure

Objective: Assess the Database Schema challenge and understand how an SQL injection weakness could expose database structure or information unavailable to an ordinary user.

Testing record: The challenge was selected and investigated in the local Juice Shop environment. The user reported that the challenge had been completed during practical work.

Outcome: Completion is user-reported. The retained screenshot shows the Database Schema challenge card only; it does not show a payload, returned schema data, or a success notification. The available evidence therefore does not independently verify successful exploitation.

Security significance: Unsafe construction of SQL statements from user-controlled input can allow query logic to be altered. Depending on query design and database permissions, this may expose schema details or data and may enable more serious database actions.

Recommended remediation: Use parameterized queries or prepared statements; avoid SQL string concatenation; apply least-privilege database permissions; return generic errors to users; and add SQL injection regression tests.

Evidence status: Incomplete — challenge selection is evidenced; exploit result is not.

3.2 View Basket — Broken Access Control

Objective: Investigate whether basket information or basket operations could be accessed outside the authorization boundaries intended by the application.

Testing record: The basket page was opened, browser Network activity associated with product quantities and basket-related requests was inspected, and client-side event listeners were examined.

Outcome: The investigation was completed, but the retained evidence does not demonstrate that one user's basket was successfully accessed or modified by another unauthorized user. The observed requests and event listeners are diagnostic evidence, not proof of a broken access-control vulnerability.

Security significance: If an API relies on client-supplied identifiers without verifying ownership server-side, one user may be able to view or manipulate another user's basket.

Recommended remediation: Derive identity from the server-side session or token; verify basket ownership on every read, update, and delete; enforce authorization at the API layer; and test cross-account access using two separate accounts.

Evidence status: Investigation documented; no successful unauthorized-access exploit demonstrated.

3.3 DOM-Based Cross-Site Scripting — DOM XSS

Objective: Determine whether attacker-controlled data processed by client-side JavaScript could be interpreted as executable content in the browser.

Testing record: A crafted value was supplied through the application's search URL. The input reached a client-side execution context and triggered a browser alert.

Outcome: Successful demonstration. The retained alert screenshot shows JavaScript execution in the tested local application context.

Security impact: DOM XSS can execute injected script in the vulnerable site's browser context. Depending on application behavior and protections, this may expose page data or enable actions as the victim.

Recommended remediation: Avoid inserting untrusted input into dangerous DOM sinks such as innerHTML; prefer textContent or safe DOM construction; validate and encode data for the correct output context; and use a restrictive Content Security Policy as defense in depth.

Evidence status: Direct screenshot evidence of alert execution.

3.4 Reflected Cross-Site Scripting — Reflected XSS

Objective: Test whether a value supplied in a URL parameter is reflected into the resulting page in an executable HTML context.

Testing record: A benign HTML heading marker was submitted through the search parameter and rendered as a heading. A follow-up crafted input triggered a browser alert with the message “Reflected XSS” during the session.

Outcome: Successful demonstration was reported and observed during the session. The retained screenshots directly show HTML reflection and the challenge hint, but do not capture the JavaScript alert itself.

Security impact: Reflected XSS occurs when request data is returned in a page without suitable output encoding or sanitization. A crafted link may cause script execution when opened by another user.

Recommended remediation: Apply context-aware output encoding; treat URL parameters as untrusted; avoid inserting reflected data with innerHTML; use a restrictive Content Security Policy as defense in depth; and add regression tests for search parameters.

Evidence status: Partial — HTML reflection is shown; the alert is reported but not separately captured.

3.5 Password Strength — Authentication and Password Security

Objective: Complete the Password Strength challenge and consider risks created by weak or compromised passwords.

Testing record: The challenge was completed in the local application. The application displayed a green success notification for Password Strength. A separate Login Admin notification was also visible; this report records only the Password Strength result supported by its own banner.

Outcome: Successful challenge completion confirmed by the application success notification.

Security significance: Weak, reused, or previously compromised passwords increase account-takeover risk.

Recommended remediation: Apply sensible password requirements; screen new passwords against known-compromised password lists where feasible; rate-limit authentication attempts; support multi-factor authentication; encourage unique passwords and password managers; and store passwords using a modern adaptive hashing algorithm with unique salts.

Evidence status: Direct screenshot evidence of the challenge success notification.

3.6 Exposed Credentials — Sensitive Data Exposure

Objective: Inspect client-side application code for credentials that should not be distributed to users' browsers.

Testing record: Browser developer tools were used to inspect the bundled application JavaScript. The inspected source contained a testing account username and password embedded in the client-side login component. The lab credentials are intentionally not reproduced in this report.

Outcome: Successful challenge completion confirmed. The source inspection identified the client-side testing credentials, and the application displayed its success notification.

Security impact: Anything shipped to a browser should be treated as public. Hardcoded credentials in client-side bundles can be extracted by users and may enable unauthorized access if valid beyond a deliberately isolated test account.

Recommended remediation: Never embed passwords, API secrets, or privileged credentials in frontend code; keep secrets in protected server-side configuration or a secrets manager; rotate exposed secrets; use low-privilege test accounts; and include secret scanning in CI/CD.

Evidence status: Direct screenshot evidence of source inspection and challenge success. Lab credential details are redacted in this report.

4. Overall Recommendations

The following controls address the recurring themes identified across the six challenges:

• **Input handling:** Treat all client-controlled values, including URL parameters and form input, as untrusted. Validate input and apply context-aware output encoding.

• **Safe rendering:** Avoid dangerous DOM and HTML sinks such as innerHTML for untrusted content. Prefer safe text APIs and DOM construction.

• **Database security:** Use parameterized queries and least-privilege database accounts.

• **Authorization:** Enforce ownership and access checks on the server for every sensitive API operation.

• **Credential management:** Keep secrets out of frontend bundles, rotate exposed credentials, and scan source/build artifacts for secrets.

• **Authentication hardening:** Use adaptive password hashing, rate limiting, multi-factor authentication, and compromised-password screening where feasible.

• **Verification and logging:** Add automated regression tests and monitor security-relevant events without logging secrets or unnecessary sensitive data.

5. Conclusion

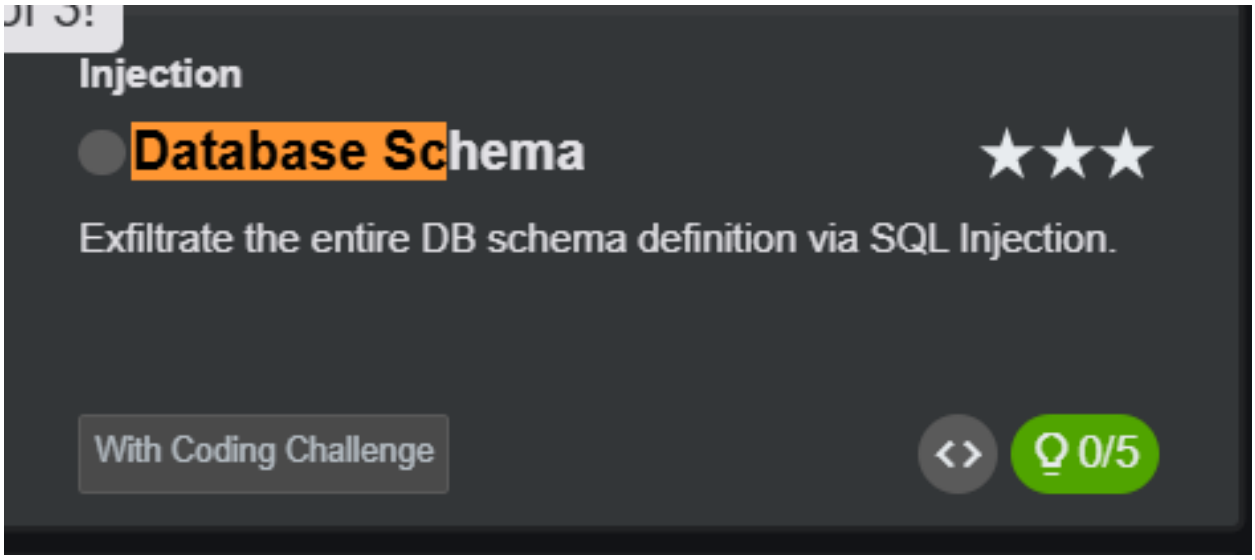
The practical assessment provided hands-on experience with common web application security weaknesses in an intentionally vulnerable local environment. Direct evidence supports successful DOM XSS execution, Password Strength challenge completion, and Exposed Credentials challenge completion. Reflected XSS was observed, with the retained evidence showing HTML reflection but not the subsequent alert. Database Schema completion was reported but is not independently established by the retained screenshot. View Basket investigation was completed without demonstrating unauthorized cross-account access.

The principal reporting conclusion is that evidence quality matters: a challenge card, a network request, or an event-listener inspection can document testing activity, but does not by itself prove exploitability. The assessment records these limitations rather than overstating the results.

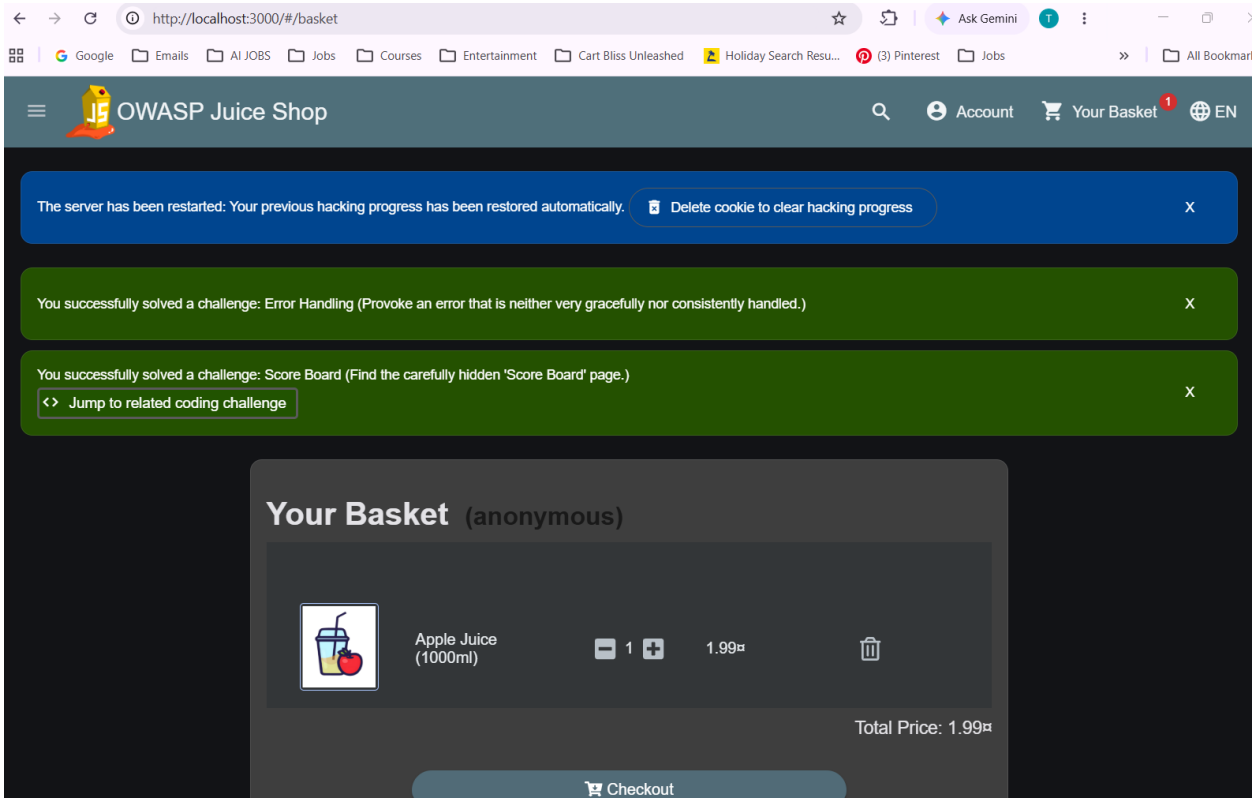
6. Evidence Appendix

The following images are selected from the supplied evidence archive. They are reproduced as supporting evidence; captions describe only what each image supports.

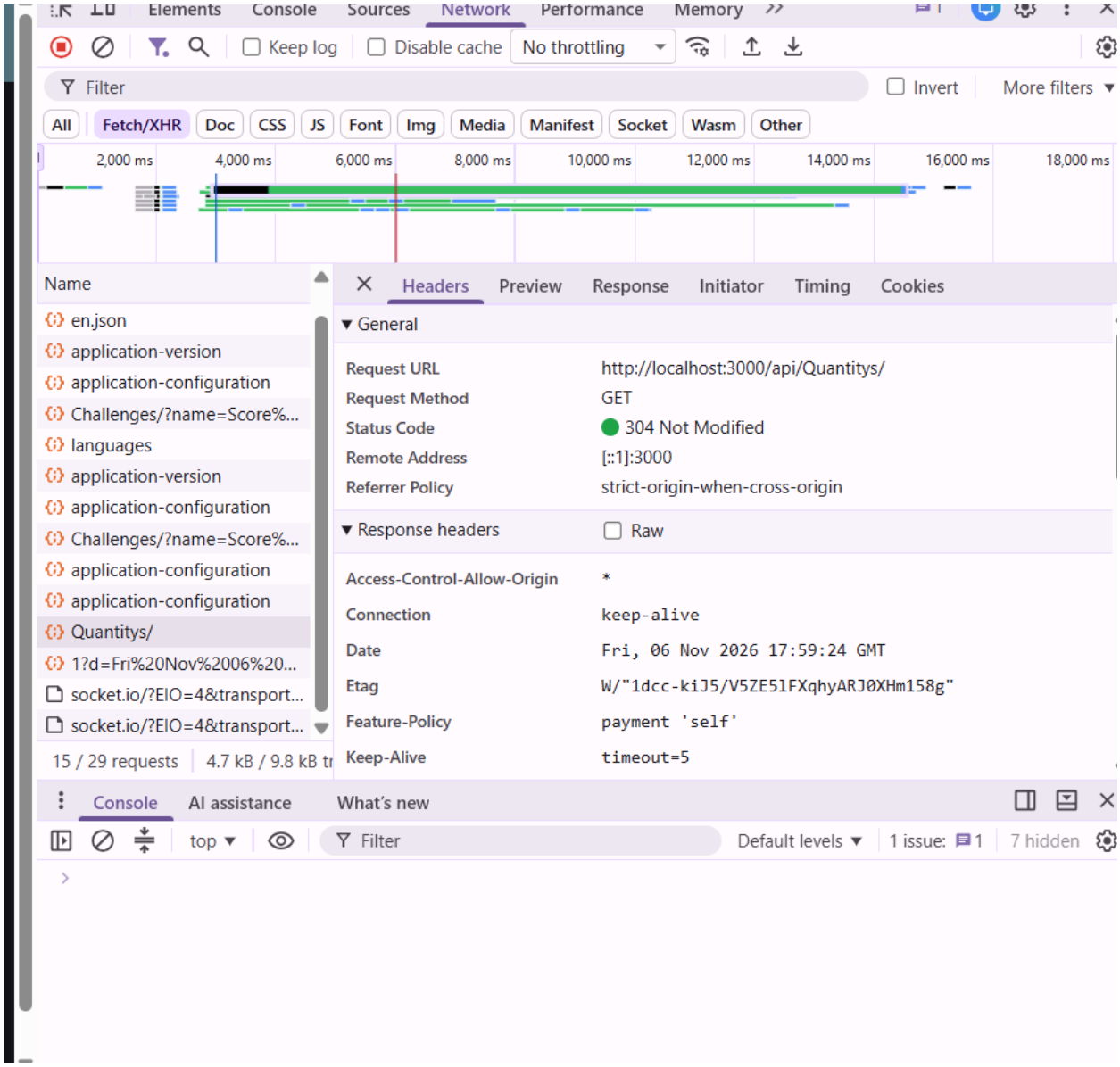
Database Schema — challenge card. Supports challenge selection only; it does not show exploit success.



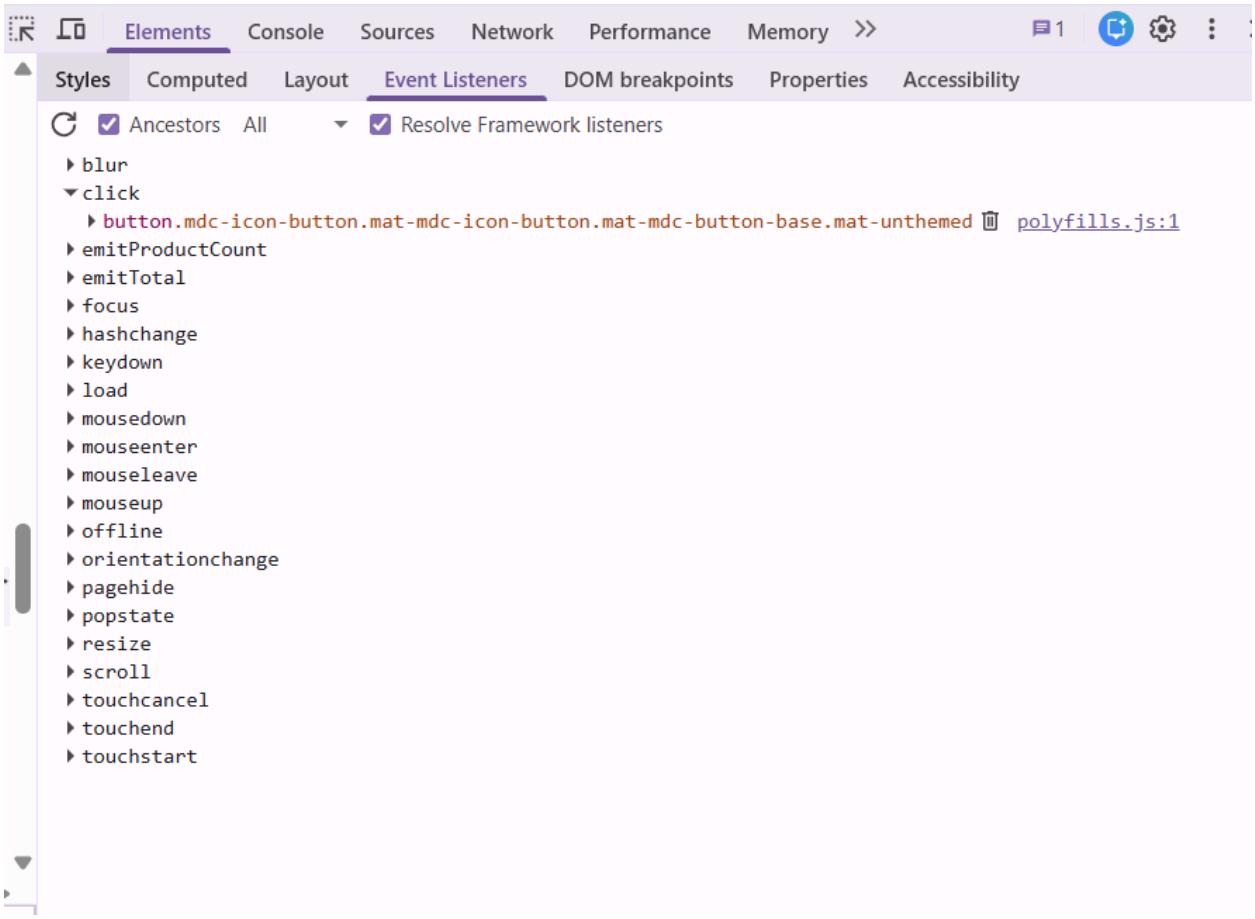
View Basket — basket page observed during the investigation.



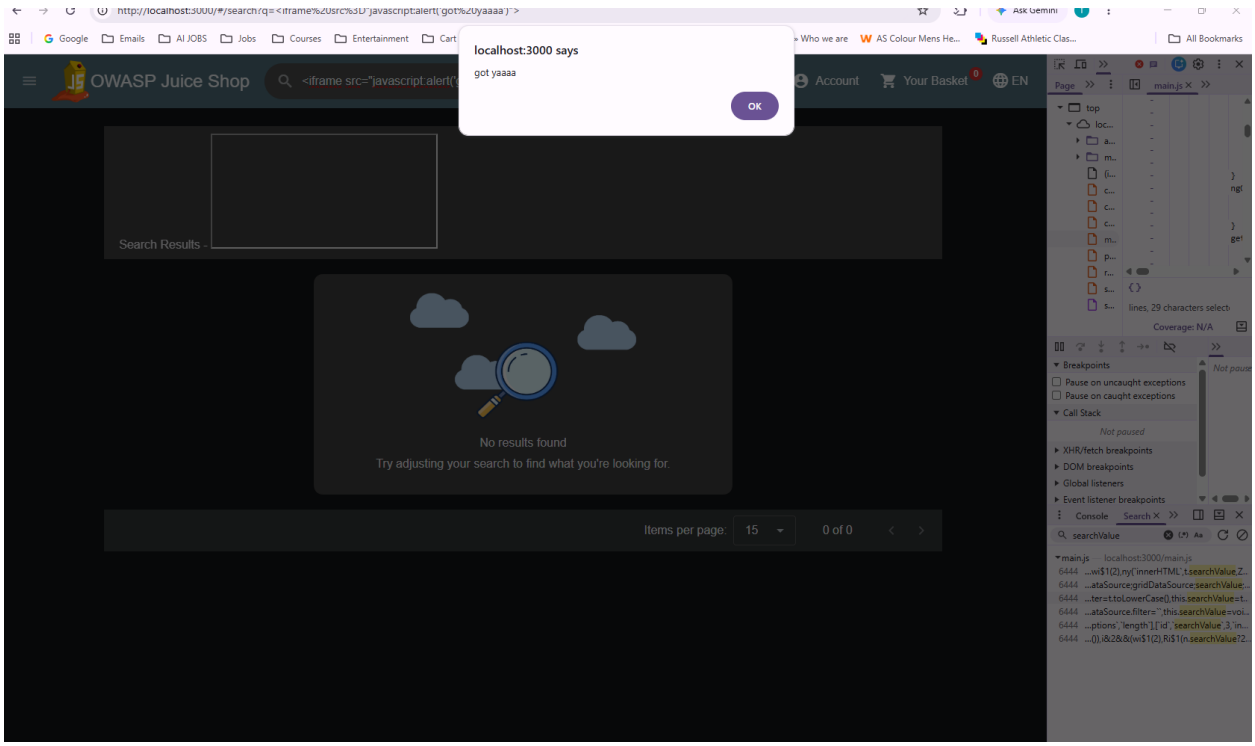
View Basket — browser Network inspection of the quantities request.



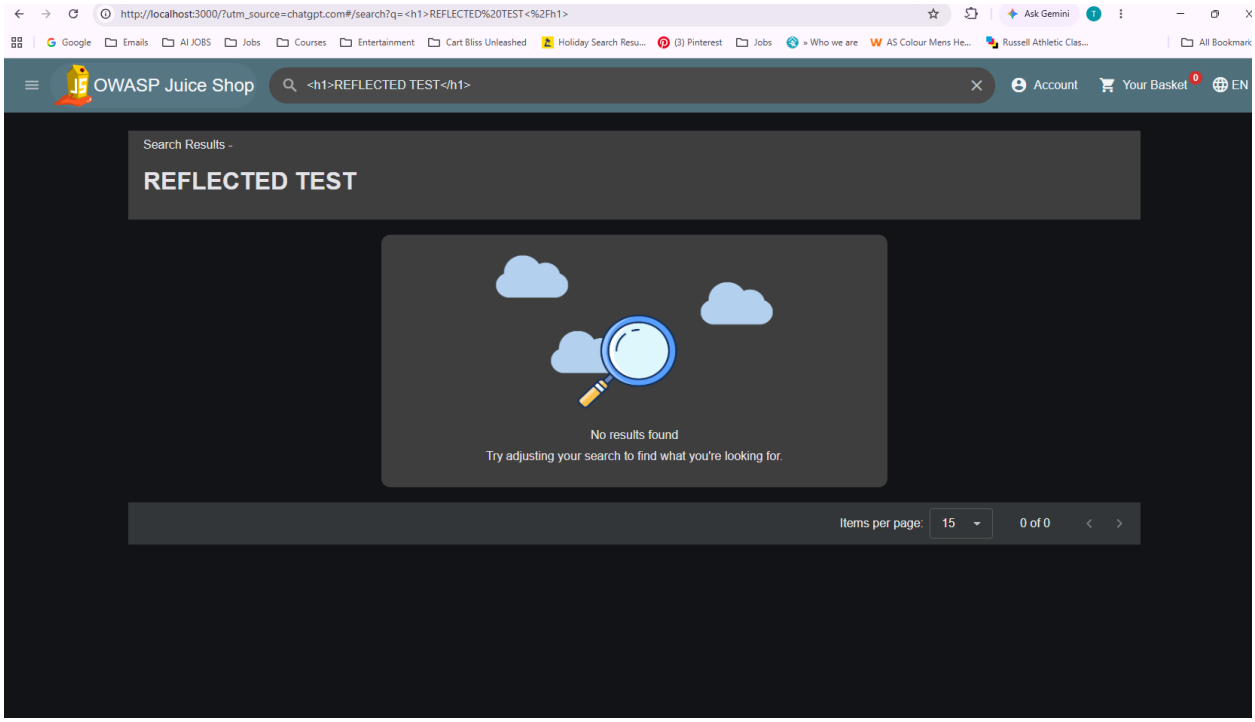
View Basket — client-side event-listener inspection.



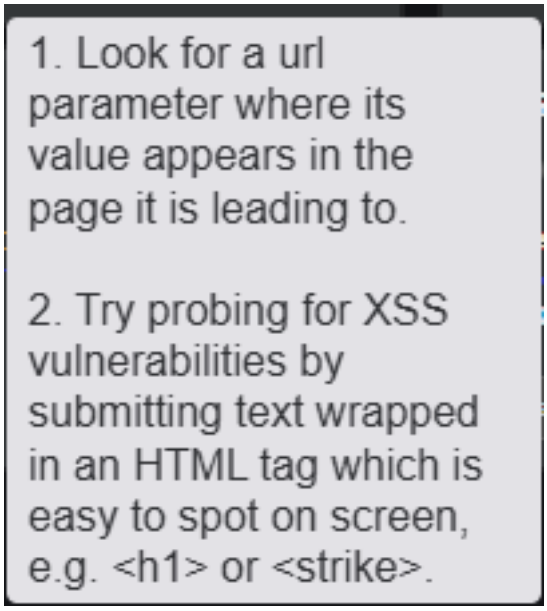
DOM XSS — browser alert displayed after crafted input was processed.



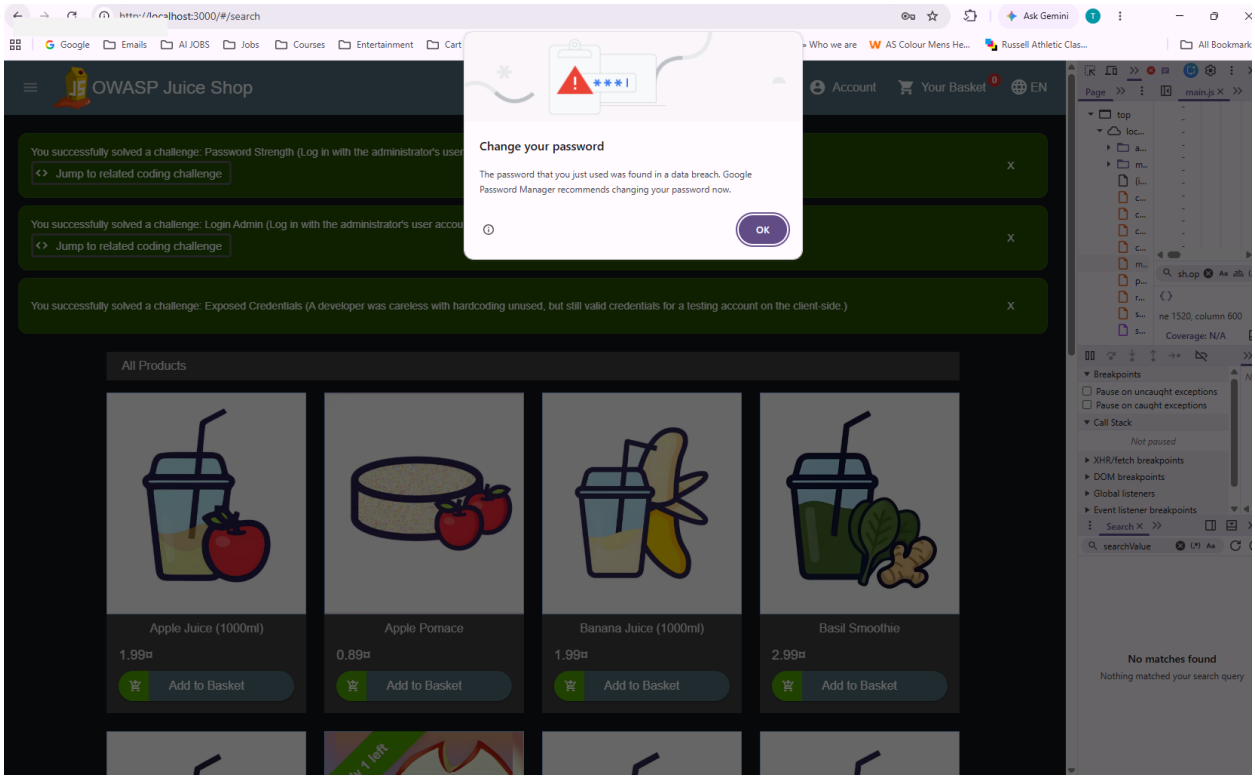
Reflected XSS — HTML heading markup rendered in the search results page.



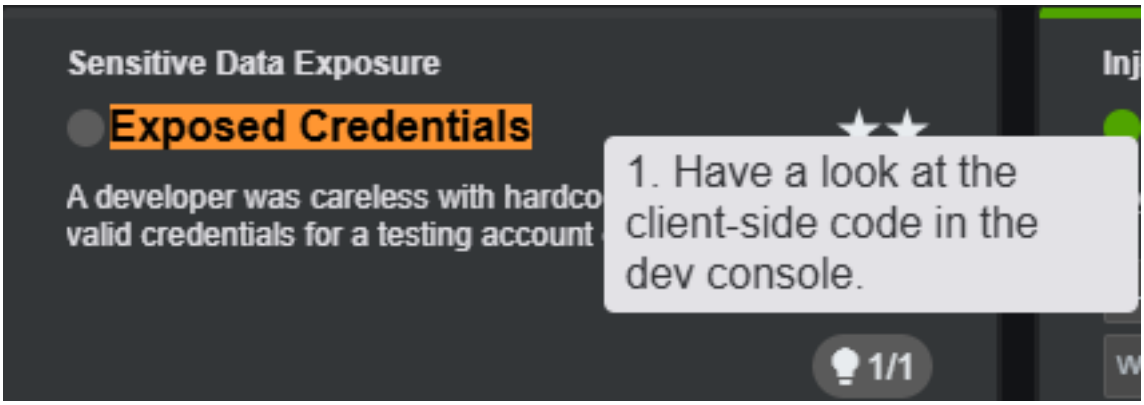
Reflected XSS — challenge guidance referring to URL parameters and HTML-tag probing.



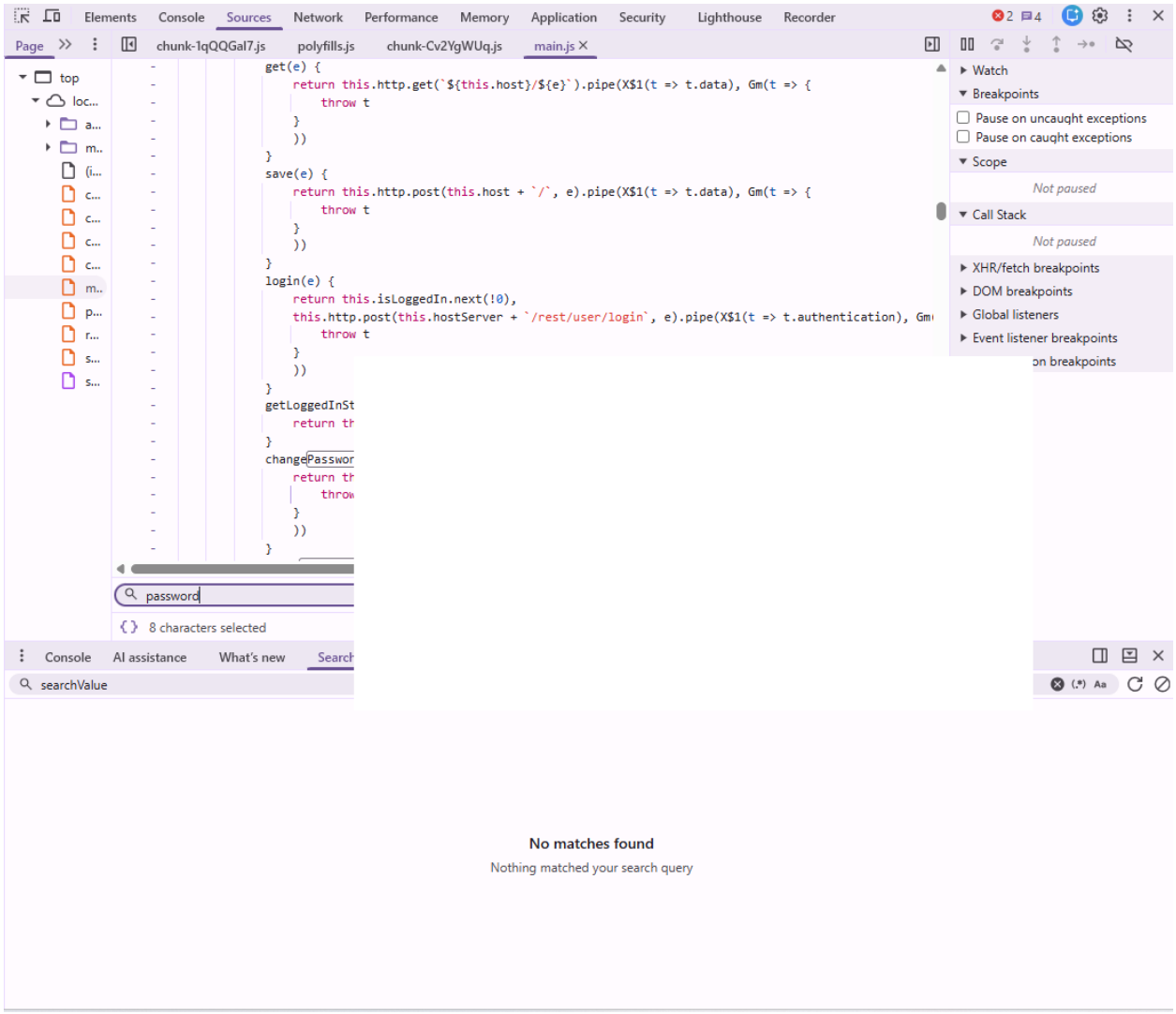
Password Strength — application success notification.



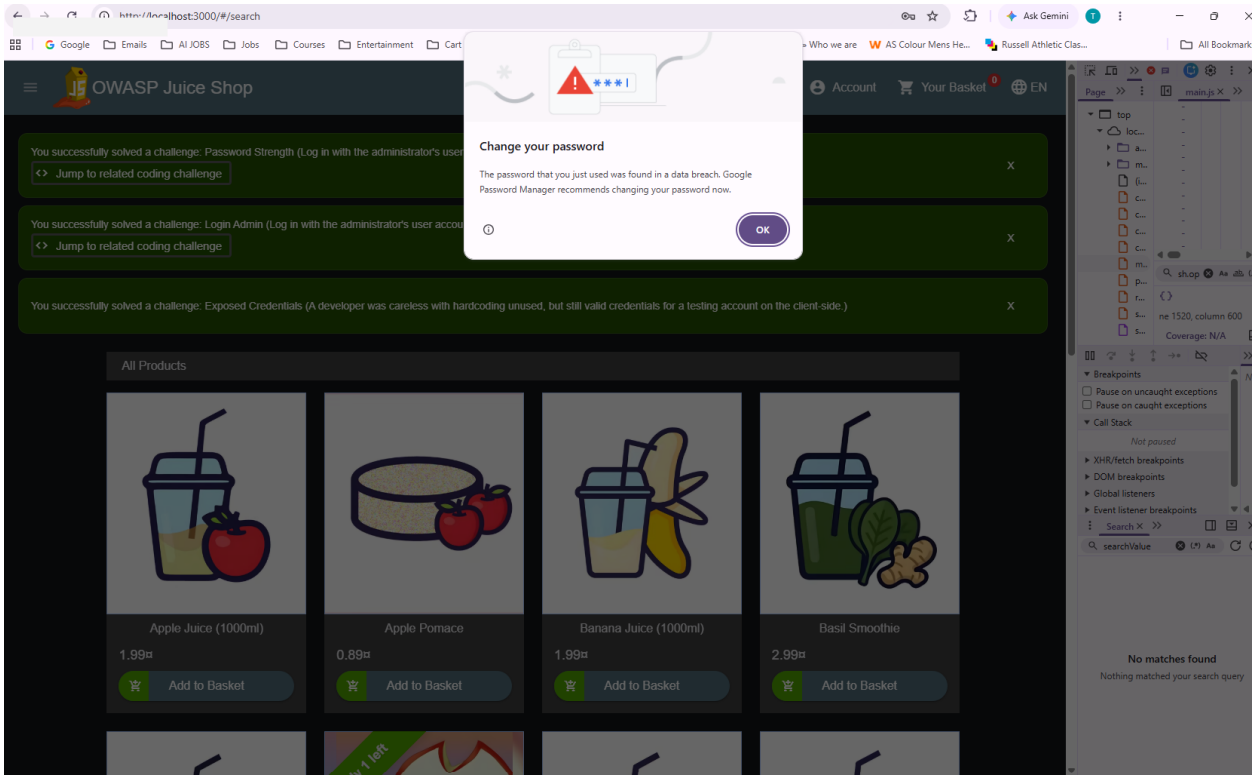
Exposed Credentials — challenge card.



Exposed Credentials — client-side source inspection. Credential text has been redacted in this report.



Exposed Credentials — application success notification.


OWASP Juice Shop
Account
Your Basket
EN
Change your password
The password that you just used was found in a data breach. Google Password Manager recommends changing your password now.
OK
You successfully solved a challenge: Password Strength (Log in with the administrator's user
Jump to related coding challenge
You successfully solved a challenge: Login Admin (Log in with the administrator's user accou
Jump to related coding challenge
You successfully solved a challenge: Exposed Credentials (A developer was careless with hardcoding unused, but still valid credentials for a testing account on the client-side.)
All Products
Apple Juice (1000ml)
1.99¤
Add to Basket
Apple Pomace
0.89¤
Add to Basket
Banana Juice (1000ml)
1.99¤
Add to Basket
Basil Smoothie
2.99¤
Add to Basket
No matches found
Nothing matched your search query

Appendix B — Evidence Limitations

The evidence archive contains 11 image files plus an evidence index. It does not contain a separate browser-alert screenshot for Reflected XSS or a success/result screenshot for Database Schema. It also contains no successful cross-account access evidence for View Basket. Those gaps are reflected in the finding outcomes and should remain visible in any portfolio presentation.

Appendix C — Safe Handling Note

The Exposed Credentials challenge used credentials embedded in the intentionally vulnerable Juice Shop training application. The credentials are not repeated in this report. Do not use training credentials against any other system. Keep the original evidence archive in a controlled portfolio folder and avoid publishing screenshots that reveal credentials unless they have been redacted.